管理员 API

Keystone

NetApp
February 15, 2022

This PDF was generated from https://docs.netapp.com/zh-cn/keystone/seapiref_administrator_apis_overview.html on February 15, 2022. Always check docs.netapp.com for the latest.

目录

管理员 API . . . 1
- 概述 . . . 1
- Tenants . . . 1
- 子租户 . . . 9
- 用户 . . . 13
- 分区 . . . 22
- regions . . . 26
- ONTAP 集群 . . . 31
- StorageGRID 实例 . . . 45
- 服务级别 . . . 51
- 服务请求 . . . 61
- 作业 . . . 67

管理员 API

概述

本节介绍以下管理员 API：

- Tenants
- 子租户
- 用户

  在使用者 API 套件中，可以执行重置用户密码，刷新令牌或以用户身份登录等活动。请参见（使用者）管理 API 。

- 分区
- regions
- ONTAP 集群
- StorageGRID 实例
- 服务级别
- 服务请求
- 作业

Tenants

使用下表中列出的方法检索，创建，修改和删除租户。

HTTP 方法	路径	**Description**
获取	` v2.1/tenants`	检索所有租户的列表。
获取	` /v2.1/teners/ ｛ id ｝ `	按租户 ID 检索租户。
发布	` v2.1/tenants`	创建新租户。
`PUT`	` /v2.1/teners/ ｛ id ｝ `	修改租户的详细信息。
d删除	` /v2.1/teners/ ｛ id ｝ `	删除租户。

租户属性

下表列出了租户属性。

属性	**Type**	**Description**
`id`	string	租户的唯一标识符。
代码	string	客户指定的（或默认的）代码，用于表示租户。此属性可以包含小写字母，数字和下划线。

属性	Type	Description
名称	string	租户名称。
`zuora_account_name`	string	计费帐户名称： Zuora 中订阅的名称。
`zuora_account_number`	string	计费帐户编号： Zuora 中的订阅编号。
问题描述`	string	租户的问题描述。
使用情况	–	适用于租户的服务和服务详细信息。对于每个服务级别，此属性将显示以下内容： * 名称： * 服务级别名称 * 已用 _size_GB ： * 服务级别名称 * 角色名称： * 用户角色（用户，管理员，读取，配对或 root ）

检索所有租户

使用下表中列出的方法检索所有租户或所有租户的一部分。

HTTP 方法	路径	Description	Parameters
获取	` v2.1/tenants`	检索所有租户。	`offset` 和 `limit` —请参见 "通用分页"

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 2 records.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 2,
    "total_records": 23,
    "sort_by": "created",
    "order_by": "desc",
    "offset": 0,
    "limit": 2,
    "records": [
```

```
      {
        "id": "5e7c3af7aab46c00014ce877",
        "name": "MyTenant",
        "zuora_account_name": "MyAccount",
        "zuora_account_number": "A00000415",
        "description": "",
        "code": "mytenantcode",
        "usage": {
          "A-S00003875": [
            {
              "service_level": "extreme",
              "consumed": 0,
              "committed": 10,
              "burst": 0
            },
            {
              "service_level": "standard",
              "consumed": 1.94,
              "committed": 30,
              "burst": 0
            }
          ],
          "A-S00004566": [
            {
              "service_level": "object",
              "consumed": 3.31,
              "committed": 300,
              "burst": 0
            }
          ]
        }
      },
      {
        "id": "5d914499869caefed0f39eee",
        "name": "MyOrg",
        "zuora_account_name": "MyOrg Inc",
        "zuora_account_number": "A00000415",
        "description": "",
        "code": "myorg",
        "usage": {
          "A-S00003875": [
            {
              "service_level": "standard",
              "consumed": 12.33,
              "committed": 30,
              "burst": 0
```

```
            },
            {
              "service_level": "object",
              "consumed": 0,
              "committed": 40,
              "burst": 0
            }
          ],
          "A-S00003969": [
            {
              "service_level": "extreme",
              "consumed": 0,
              "committed": 5,
              "burst": 0
            }
          ]
        }
      }
    ]
  }
}
```


按 **ID** 检索租户

使用下表中列出的方法按 ID 检索租户。

HTTP 方法	路径	**Description**	**Parameters**
获取	`/v2.1/teners/｛id｝`	检索 ID 指定的租户。	`id` （ `string` ）：租户的唯一标识符。

所需的请求正文属性： `none`

请求正文示例：

```
none
```

- 响应正文示例： *


```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
```

```
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5e7c3af7aab46c00014ce877",
        "name": "MyTenant",
        "zuora_account_name": "MyAccount",
        "zuora_account_number": "A00000415",
        "description": "",
        "code": "mytenantcode",
        "usage": {
          "A-S00003875": [
            {
              "service_level": "extreme",
              "consumed": 0,
              "committed": 10,
              "burst": 0
            },
            {
              "service_level": "premium",
              "consumed": 2.4,
              "committed": 20,
              "burst": 0
            },
            {
              "service_level": "standard",
              "consumed": 1.94,
              "committed": 30,
              "burst": 0
            },
            {
              "service_level": "object",
              "consumed": 0,
              "committed": 40,
              "burst": 0
            }
          ],
          "A-S00003969": [
            {
              "service_level": "extreme",
              "consumed": 0,
              "committed": 5,
              "burst": 0
            },
            {
              "service_level": "standard",
```

```
                "consumed": 0,
                "committed": 30,
                "burst": 0
              }
            ],
            "A-S00004566": [
              {
                "service_level": "object",
                "consumed": 3.31,
                "committed": 300,
                "burst": 0
              }
            ]
          }
        }
      ]
    }
  }
```


创建租户

使用下表中列出的方法创建租户。

HTTP 方法	路径	Description	Parameters
发布	` v2.1/tenants`	创建新租户。	无

所需请求正文属性： `code` ， `name` ， `zuora_account_name` ， `zuora_account_number`

- 请求正文示例： *


```
{
  "name": "MyNewTenant",
  "code": "mytenant",
  "zuora_account_name": "string",
  "zuora_account_number": "A00000415",
  "description": "DescriptionOfMyTenant"
}
```


- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. New resource created.",
    "verbose_message": "",
    "code": 201
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ed5ac802c356a0001a735af",
        "name": "MyNewTenant",
        "zuora_account_name": "string",
        "zuora_account_number": "A00000415",
        "description": "DescriptionOfMyTenant",
        "code": "mytenant",
        "usage": null
      }
    ]
  }
}
```

修改租户

使用下表中列出的方法修改租户。

HTTP 方法	路径	Description	Parameters
PUT	` /v2.1/teners/ ｛id｝ `	修改 ID 指定的租户。您可以更改租户的名称，Zuora 订阅详细信息（帐户名称或订阅编号）和问题描述。	id （ string ）：租户的唯一标识符。

所需的请求正文属性： code

- 请求正文示例： *

```
{
  "name": "MyNewTenant",
  "code": "mytenant",
  "zuora_account_name": "string",
  "zuora_account_number": "A00000415",
  "description": "New description of my tenant"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ed5ac802c356a0001a735af",
        "name": "MyNewTenant",
        "zuora_account_name": "string",
        "zuora_account_number": "A00000415",
        "description": "New description of my tenant",
        "code": "mytenant",
        "usage": null
      }
    ]
  }
}
```

删除租户

使用下表中列出的方法删除租户。

HTTP 方法	路径	Description	Parameters
d删除	` /v2.1/teners/ ｛id｝ `	删除此 ID 指定的租户。	id （ string ）：租户的唯一标识符。

所需的请求正文属性： none

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
No content for successful delete
```

子租户

使用下表中列出的方法检索，创建，修改和删除子租户。

HTTP 方法	路径	Description
获取	` /v2.1/ 子租户`	检索子租户。
获取	` /v2.1/subteners/ ｛ id ｝ `	按子租户 ID 检索子租户。
发布	` /v2.1/ 子租户``	创建新的子租户。
`PUT`	` /v2.1/subteners/ ｛ id ｝ `	修改子租户的详细信息。您可以修改子租户的名称。
`d`删除	` /v2.1/subteners/ ｛ id ｝ `	删除子租户。

子租户属性

下表列出了子租户属性。

属性	Type	Description
`id`	string	子租户的唯一标识符。
名称	string	子租户名称。
代码	string	客户指定的（或默认的）代码，表示子租户。
`tenant_id`	string	子租户所属租户的标识符。

检索所有子租户

使用下表中列出的方法检索所有子租户或所有子租户的一部分。指定 `tenant_id` 将仅返回属于该租户的子租户。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/ 子租户`	检索子租户。	`tenant_id`：（可选）返回属于指定租户的子租户。偏移和限制—请参见

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 2 records.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 2,
    "total_records": 202,
    "sort_by": "created",
    "order_by": "desc",
    "offset": 0,
    "limit": 2,
    "records": [      {
        "id": "5e7c3af8aab46c00014ce878",
        "description": "",
        "name": "MySubtenant",
        "code": "mysubtenant",
        "tenant_id": "5e7c3af7aab46c00014ce877",
        "tenant": "MyTenant"
      },
      {
        "id": "5d9144f3869caefed0f39f82",
        "description": "",
        "name": "Mysubtenant2",
        "code": "myothersubtenant",
        "tenant_id": "5d914499869caefed0f39eee",
        "tenant": "MyTenant"
      }
    ]
  }
}
```


按 **ID** 检索子租户

使用下面列出的方法按 ID 检索子租户。

HTTP 方法	路径	**Description**	**Parameters**
获取	`/v2.1/subteners/｛id｝`	检索 ID 指定的子租户。	`id` （ `string` ）：子租户的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5e7c3af8aab46c00014ce878",
        "description": "",
        "name": "MySubtenant",
        "code": "subtenantcode",
        "tenant_id": "5e7c3af7aab46c00014ce877",
        "tenant": "MyTenant"
      }
    ]
  }
}
```

创建子租户

使用下表中列出的方法创建子租户。

HTTP 方法	路径	Description	Parameters
发布	` /v2.1/ 子租户`	创建新的子租户。	无

所需请求正文属性： `name` ， `code` ， `tenant_id`

- 请求正文示例： *

```
{
  "name": "MySubtenant",
  "code": "mynewsubtenant",
  "tenant_id": "5ed5ac802c356a0001a735af"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. New resource created.",
    "verbose_message": "",
    "code": 201
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ecefbbef418b40001f20bd6",
        "description": "",
        "name": "MyNewSubtenant",
        "code": "mynewsubtenant",
        "tenant_id": "5e7c3af7aab46c00014ce877",
        "tenant": "MyTenant"
      }
    ]
  }
}
```

按 **ID** 修改子租户

使用下表中列出的方法按 ID 修改子租户。

HTTP 方法	路径	**Description**	**Parameters**
`PUT`	`/v2.1/subteners/{id}`	修改 ID 指定的子租户。您可以更改子租户名称。	`id` （ `string` ）：子租户的唯一标识符。

所需的请求正文属性： `name`

- 请求正文示例： *

```
{
  "name": "MyModifiedSubtenant"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ecefbbef418b40001f20bd6",
        "description": "",
        "name": "MyNewSubtenant",
        "code": "mynewsubtenant",
        "tenant_id": "5e7c3af7aab46c00014ce877",
        "tenant": "MyTenant"
      }
    ]
  }
}
```

按 **ID** 删除子租户

使用下表中列出的方法按 ID 删除子租户。

HTTP 方法	路径	**Description**	**Parameters**
d删除	` /v2.1/subteners/ ｛id｝ `	删除此 ID 指定的子租户。	`id` （ `string` ）：子租户的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
No content for succesful delete
```

用户

使用下表中列出的方法检索，创建，修改和删除子租户。

HTTP 方法	路径	Description
获取	` v2.1/users`	检索所有用户的列表。
获取	` /v2.1/users/ ｛ id ｝ `	按用户 ID 检索用户。
发布	` v2.1/users`	创建新用户：
PUT	` /v2.1/users/ ｛ id ｝ `	修改用户的详细信息。
d删除	` /v2.1/users/ ｛ id ｝ `	删除用户。
获取	` /v2.1/Users/ ｛ username ｝ `	按用户名检索用户。

用户属性

下表列出了用户属性。

属性	Type	Description
id	string	用户的唯一标识符。
用户名	string	用户名。
密码	string	用户的密码。
名字	string	用户的名字。
lastName	string	用户的姓氏。
displayName	string	用户的显示名称。
电子邮件	string	用户的电子邮件地址。
电话	string	用户的电话号码。
profileImageURL	string	用户配置文件映像的 URL 。
tenant_id	string	此用户的主租户标识符。
租约	–	用户所属的租户；一个数组，包含： • tenant_id ，和 • role_name 这是用户角色；用户，管理员，读取，配对， 或 root 。
提供程序	string	身份验证提供程序： local 或 ActiveDirectory
提供程序数据	–	身份验证提供程序详细信息，包括： • EMail_address ，和 • m成员 （组成员资格）

检索所有用户

使用下表中列出的方法检索所有用户。

HTTP 方法	路径	Description	Parameters
获取	` v2.1/users`	检索所有用户。	无

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5dddbe0ef071fe0001b889fd",
        "username": "TestUser3",
        "firstName": "Test",
        "lastName": "User",
        "displayName": "",
        "email": "testuser@netapp.com",
        "tenancies": [
          {
            "id": "5d914499869caefed0f39eee",
            "name": "MyOrg",
            "code": "myorg",
            "role": "admin"
          }
        ]
      }
    ]
  }
}
```

按 **ID** 检索用户

使用下表中列出的方法按 ID 检索用户。

HTTP 方法	路径	**Description**	**Parameters**
获取	` v2.1/users`	按 ID 检索用户。	`id` （ `string` ）：用户的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5e585df6896bd80001dd4b44",
        "username": "testuser01",
        "firstName": "",
        "lastName": "",
        "displayName": "",
        "email": "",
        "tenancies": [
          {
            "id": "5d914499869caefed0f39eee",
            "name": "MyOrg",
            "code": "myorg",
            "role": "user"
          }
        ]
      }
    ]
  }
}
```

按用户名检索用户

使用下表中列出的方法按用户名检索用户。

HTTP 方法	路径	Description	Parameters
获取	` v2.1/users`	按用户名检索用户。	`username` （ `string` ）：用户的用户名。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5e61aa814559c20001df1a5f",
        "username": "MyName",
        "firstName": "MyFirstName",
        "lastName": "MySurname",
        "displayName": "CallMeMYF",
        "email": "user@example.com",
        "tenancies": [
          {
            "id": "5e5f1c4f253c820001877839",
            "name": "MyTenant",
            "code": "testtenantmh",
            "role": "user"
          }
        ]
      }
    ]
  }
}
```

创建用户

使用下表中列出的方法创建用户。

HTTP 方法	路径	Description	Parameters
发布	` v2.1/users`	创建新用户：	无

所需请求正文属性： `username` ， `tenant_id` ， `tenancies` ， `provider`

- 请求正文示例： *

```
{
  "username": "MyUser",
  "password": "mypassword",
  "firstName": "My",
  "lastName": "User",
  "displayName": "CallMeMyUser",
  "email": "user@example.com",
  "phone": "string",
  "profileImageURL": "string",
  "tenant_id": "5e7c3af7aab46c00014ce877",
  "tenancies": [
    {
      "tenant_id": "5e7c3af7aab46c00014ce877",
      "role_name": "admin"
    }
  ],
  "provider": "local",
  "provider_data": {
    "email": "user@example.com",
    "member_of": "string"
  }
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. New resource created.",
    "verbose_message": "",
    "code": 201
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ed6f463129e5d000102f7e1",
        "username": "MyUser",
        "firstName": "My",
        "lastName": "User",
        "displayName": "CallMeMyUser",
        "email": "user@example.com",
        "tenancies": [
          {
            "id": "5e7c3af7aab46c00014ce877",
            "name": "MyTenant",
            "code": "mytenantcode",
            "role_name": "admin"
          }
        ]
      }
    ]
  }
}
```


按 **ID** 修改用户

使用下表中列出的方法按用户 ID 修改用户。

HTTP 方法	路径	**Description**	**Parameters**
`PUT`	`/v2.1/users/{id}`	修改由用户 ID 标识的用户。您可以修改用户名，显示名称，密码，电子邮件地址，电话号码， 配置文件映像 URL 和租户详细信息。	`id` （ `string` ）：用户的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
{
  "password": "MyNewPassword",
   "firstName": "MyFirstName",
   "lastName": "MySurname",
   "displayName": "CallMeMYF",
   "email": "user@example.com",
   "phone": "string",
  "profileImageURL": "string",
  "tenant_id": "5e5f1c4f253c820001877839",
  "tenancies": [
    {
      "tenant_id": "5e5f1c4f253c820001877839",
      "role_name": "user"
    }
  ]
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5e61aa814559c20001df1a5f",
        "username": "MyName",
        "firstName": "MyFirstName",
        "lastName": "MySurname",
        "displayName": "CallMeMYF",
        "email": "user@example.com",
        "tenancies": [
          {
            "id": "5e5f1c4f253c820001877839",
            "name": "MyTenant",
            "code": "testtenantmh",
            "role": "user"
          }
        ]
      }
    ]
  }
}
```

按 **ID** 删除用户

使用下表中列出的方法按 ID 删除用户。

HTTP 方法	路径	**Description**	**Parameters**
d删除	`/v2.1/users/{name}`	删除此 ID 标识的用户。	`id` （ `string` ）：用户的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
No content for succesful delete
```

分区

使用下表中列出的方法创建，修改和删除分区。有关可用于检索区域的 API ，请参见（使用者）管理 API 。

HTTP 方法	路径	Description
发布	` v2.1/zones`	创建新分区。
`PUT`	` /v2.1/zones/ ｛ name ｝ `	修改分区的详细信息。
d删除	` /v2.1/zones/ ｛ name ｝ `	删除分区。

分区属性

下表列出了分区属性。

属性	Type	Description
`id`	string	分区的唯一标识符。
名称	string	分区名称。
问题描述`	string	分区的问题描述。
`reGion_name`	string	分区所在区域的名称。

检索所有分区

使用下表中列出的方法检索所有分区或部分分区。指定区域将仅返回属于该租户的块存储。

HTTP 方法	路径	Description	Parameters
获取	` v2.1/zones`	检索分区。	（可选）区域名称（字符串） `offset` 和 `limit` — 请参见 "通用分页"

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "total_records": 5,
    "sort_by": "created",
    "order_by": "desc",
    "offset": 2,
    "limit": 1,
    "records": [
      {
        "id": "5ce5d919b68d3b82dc34bef2",
        "name": "au-west1-a",
        "description": "au-west1-a",
        "region": "au-west1"
      }
    ]
  }
```

按名称检索分区

使用下表中列出的方法按分区名称检索分区。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/zones/ ｛name｝ `	按名称检索分区。	`Name` （ `string` ）：分区名称

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5ce5d919b68d3b82dc34bef2",
        "name": "au-west1-a",
        "description": "au-west1-a",
        "region": "au-west1"
      }
    ]
  }
}
```

创建分区

使用下表中列出的方法创建分区。

HTTP 方法	路径	Description	Parameters
发布	` v2.1/zones`	在区域中创建新分区。	无

所需的请求正文属性： `name` ，问题描述` ， `reGion_name`

- 请求正文示例： *

```
{
  "name": "MyZoneName",
  "description": "DescriptionOfMyZone",
  "region_name": "MyRegionName"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. New resource created.",
    "verbose_message": "",
    "code": 201
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5e61741c9b64790001fe9663",
        "name": "MyZoneName",
        "description": "DescriptionOfMyZone",
        "region": "MyRegionName"
      }
    ]
  }
}
```

修改分区

使用下表中列出的方法修改分区。

HTTP 方法	路径	Description	Parameters
PUT	`/v2.1/zones｛name｝`	修改以名称标识的分区。	name （ string ）：分区的名称。

所需的请求正文属性： none

- 请求正文示例： *

```
{
  "name": "MyZoneName",
  "description": "NewDescriptionOfMyZone"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5e61741c9b64790001fe9663",
        "name": "MyZoneName",
        "description": "NewDescriptionOfMyZone",
        "region": "MyRegionName"
      }
    ]
  }
}
```

删除分区

使用下表中列出的方法删除分区。

HTTP 方法	路径	Description	Parameters
d删除	` /v2.1/zones ｛name｝ `	删除通过名称标识的单个分区。必须先删除区域中的所有存储资源。	`name` （ `string` ）：分区的名称。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

成功删除后，不会返回任何内容。

regions

使用下表中列出的方法创建，修改和删除区域。有关可用于检索区域的 API ，请参见（使用者）管理 API 。

HTTP 方法	路径	Description
获取	` v2.1/Regions`	获取区域。
获取	` /v2.1/Regies/ ｛name｝ `	按名称获取区域。
发布	` v2.1/Regions`	创建新区域。
`PUT`	` /v2.1/Regies/ ｛name｝ `	修改区域的详细信息。
`d`删除	` /v2.1/Regies/ ｛name｝ `	删除区域。

区域属性

下表列出了区域属性。

属性	Type	Description
`id`	string	区域的唯一标识符。
名称	string	区域名称。
问题描述`	string	区域的问题描述。

检索区域

使用下表中列出的方法检索所有区域或部分区域。

HTTP 方法	路径	Description	Parameters
获取	` v2.1/Regions`	检索区域。	`offset` 和 `limit` —请参见 "通用分页"

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "total_records": 4,
    "sort_by": "created",
    "order_by": "desc",
    "offset": 0,
    "limit": 1,
    "records": [
      {
        "id": "5e7bf44daab46c00014ce77f",
        "name": "au-east8",
        "description": "This is the new region description",
        "zones": []
      }
    ]
  }
}
```


按名称检索区域

使用下表中列出的方法按名称检索区域。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/Regies/ ｛name ｝ `	按名称检索区域。	`name` （ `string` ）：区域名称。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "string",
    "verbose_message": "string",
    "code": "string"
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5d2fb0fb4f47df00015274e3",
        "name": "au-east1",
        "description": "string",
        "zones": [
          "au-east1-a"
        ]
      }
    ]
  }
}
```

创建区域

使用下表中列出的方法创建区域。

HTTP 方法	路径	Description	Parameters
发布	` v2.1/Regions`	创建新区域。	无

所需的请求正文属性： `name`

- 请求正文示例： *

```
{
  "name": "MyRegionName",
  "description": "DescriptionOfMyRegion"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. New resource created.",
    "verbose_message": "",
    "code": 201
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5e616f849b64790001fe9658",
        "name": "MyRegionName",
        "Description": "DescriptionOfMyRegion",
        "user_id": "5bbee380a2df7a04d43acaee",
        "created": "0001-01-01T00:00:00Z",
        "tags": null
      }
    ]
  }
}
```

修改区域

使用下表中列出的方法修改区域。

HTTP 方法	路径	Description	Parameters
PUT	`/v2.1/Regies/{name}`	修改以名称标识的区域。您可以更改区域的名称和问题描述。	name（string）：区域的名称。

所需的请求正文属性：none

- 请求正文示例：*

```
{
  "name": "MyRegionName",
  "description": "NewDescriptionOfMyRegion"
}
```

- 响应正文示例：*

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5e616f849b64790001fe9658",
        "name": "MyRegionName",
        "description": "NewDescriptionOfMyRegion",
        "zones": []
      }
    ]
  }
}
```

删除区域

使用下表中列出的方法删除区域。

HTTP 方法	路径	Description	Parameters
d删除	` /v2.1/Regions {name} `	删除使用名称标识的单个区域。必须先删除某个区域中的所有分区。	`Name` （ `string` ）：区域的名称。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
No content for succesful delete
```

ONTAP 集群

使用下表中列出的方法检索，创建，修改和删除 ONTAP 集群。

HTTP 方法	路径	Description
获取	` /v2.1/ontapclusters`	检索所有 ONTAP 集群。
获取	` /v2.1/ontapclusters/ { id } `	按 ID 检索 ONTAP 集群。
发布	` /v2.1/ontapclusters`	创建新的 ONTAP 集群。
`PUT`	` /v2.1/ontapclusters/ { id } `	按 ID 更新 ONTAP 集群清单。
`d`删除	` /v2.1/ontapclusters/ { id } `	删除 ONTAP 集群。

ONTAP 集群属性

下表列出了 ONTAP 集群的属性。

属性	Type	Description
`id`	string	ONTAP 集群的唯一标识符。
名称	string	ONTAP 集群名称。
`uuid`	string	ONTAP 集群通用唯一标识符（UUID）。
`mantioning_IP`	string	ONTAP 集群管理 IPv4 地址。
用户名	string	ONTAP 集群名称
密码	string	ONTAP 集群密码
`provisioning_state`	string	标识集群是否可用于配置操作。选项包括： • 打开 • 已关闭
`data_network_ip_cidr`	string	子网的 CIDR 表示法。
`data_network_default_gateway`	string	IPv4 地址。

属性	Type	Description

data_network_ports	–	ONTAP 集群数据网络端口列表。
		
		``
		For example:
		[
		  {
		  "node_name": "dev-ots-per01-01",
		  "port_name": "e0c-120",
		  "parent_port": "e0c"
		  }
		]
		``

intercluster_LIF	–	ONTAP 集群集群间 LIF 。
		
		``
		For example:
		[
		  {
		  "name": "peer1",
		  "node": "aff-01",
		  "port": "a0a-103",
		  "address": "10.128.113.232",
		  "netmask": "255.255.255.0"
		  },
		{
		  "name": "peer2",
		  "node": "aff-02",
		  "port": "a0a-103",
		  "address": "10.128.113.233",
		  "netmask": "255.255.255.0"
		  }
		]
		``

属性	Type	Description
sVM_root_service_level	string	ONTAP 集群 Storage Virtual Machine （ SVM ）根服务级别名称。适用值包括 " 标准 " ， " 极速 " 或 " 高级 " 。默认情况下，此服务级别会分配给在集群下创建的所有 SVM 。集群应具有所述服务级别的关联聚合。
分区	string	分区名称。
ssubscription_number	string	Zuora 订阅。
serves_available	–	列出可用服务及其状态。例如： `{ "fcp": false, "iscsi": true, "nas": true }`
data_fcp_ports	–	已启用 FCP 的 ONTAP 集群的节点和端口列表。例如： `[ { "node_name": "aff-01", "port_name": "0g" }, { "node_name": "aff-01", "port_name": "0h" ]`
is_mcc	布尔值	指示集群是否已启用 MetroCluster 。默认值为 False 。
mcc_partner_cluster	string	MetroCluster 对中当前集群的配对集群的标识符。如果为集群启用了 MetroCluster ，则为必需项。

检索所有 **ONTAP** 集群

使用以下列出的方法检索所有 ONTAP 集群或部分 ONTAP 集群。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/ontapclusters`	检索所有 ONTAP 集群。	offset 和 limit —请参见 "通用分页"

所需的请求正文属性： none

- 请求正文示例： *

```
none
```

- 响应正文示例： *


```
{
  "status": {
    "user_message": "Okay. Returned 2 records.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 2,
    "total_records": 5,
    "sort_by": "created",
    "order_by": "desc",
    "offset": 3,
    "limit": 2,
    "records": [
      {
        "id": "5c5bb9f16680a7002a5f7450",
        "name": "dev-ots-per01",
        "region": "au-west1",
        "zone": "au-west1-a",
        "uuid": "63053baa-ada4-11ea-b197-005056a4c0ef",
        "management_ip": "10.128.115.173",
        "username": "admin",
        "services_available": {
          "fcp": false,
          "iscsi": true,
          "nas": true
        },
        "provisioning_state": "open",
        "data_network_ports": [
          {
            "node_name": "dev-ots-per01-01",
            "port_name": "e0c-120",
            "parent_port": "e0c"
          }
        ],
        "data_network_ip_cidr": "10.96.120.0/24",
        "data_network_default_gateway": "10.96.120.1",
```

```
        "svm_root_service_level": "performance",
        "intercluster_lifs": [
          {
            "name": "dev-ots-per01-01-icl01",
            "node": "dev-ots-per01-01",
            "port": "e0b",
            "address": "10.128.115.144",
            "netmask": "255.255.255.0"
          }
        ],
        "subscription_number": "A-S00003875",
        "created": "2019-02-22T03:38:38.867Z",
        "data_fcp_ports": []
      },
      {
        "id": "5eaf5249f038943eb46b6608",
        "name": "aff",
        "region": "au-east1",
        "zone": "au-east1-b",
        "uuid": "62d649d2-07a1-11e6-9549-00a0985c0dcb",
        "management_ip": "10.128.113.69",
        "username": "admin",
        "services_available": {
          "fcp": true,
          "iscsi": true,
          "nas": true
        },
        "provisioning_state": "open",
        "data_network_ports": [
          {
            "node_name": "aff-01",
            "port_name": "a0a-2000",
            "parent_port": "a0a"
          },
          {
            "node_name": "aff-02",
            "port_name": "a0a-2000",
            "parent_port": "a0a"
          }
        ],
        "data_network_ip_cidr": "10.50.50.0/24",
        "data_network_default_gateway": "10.50.50.1",
        "svm_root_service_level": "premium",
        "intercluster_lifs": [
          {
            "name": "peer1",
```

```
            "node": "aff-01",
            "port": "a0a-103",
            "address": "10.128.113.232",
            "netmask": "255.255.255.0"
          },
          {
            "name": "peer2",
            "node": "aff-02",
            "port": "a0a-103",
            "address": "10.128.113.233",
            "netmask": "255.255.255.0"
          }
        ],
        "subscription_number": "A-S00004635",
        "created": "2019-02-22T03:38:38.867Z",
        "data_fcp_ports": [
          {
            "node_name": "aff-01",
            "port_name": "0g"
          },
          {
            "node_name": "aff-01",
            "port_name": "0h"
          },
          {
            "node_name": "aff-02",
            "port_name": "0g"
          },
          {
            "node_name": "aff-02",
            "port_name": "0h"
          }
        ],
        "is_mcc": false,
        "created": "1995-09-07T10:40:52Z"
      }
    ]
  }
}
```


按 **ID** 检索 **ONTAP** 集群

使用下面列出的方法按 ID 检索 ONTAP 集群。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/ontapclusters/ {id} `	检索 ID 标识的 ONTAP 集群。	`id` （ `string` ）：ONTAP 集群的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5c5bb9f16680a7002a5f7450",
        "name": "dev-ots-per01",
        "region": "au-west1",
        "zone": "au-west1-a",
        "uuid": "63053baa-ada4-11ea-b197-005056a4c0ef",
        "management_ip": "10.128.115.173",
        "username": "admin",
        "services_available": {
          "fcp": false,
          "iscsi": true,
          "nas": true
        },
        "provisioning_state": "open",
        "data_network_ports": [
          {
            "node_name": "dev-ots-per01-01",
            "port_name": "e0c-120",
            "parent_port": "e0c"
          }
        ],
        "data_network_ip_cidr": "10.96.120.0/24",
        "data_network_default_gateway": "10.96.120.1",
```

```
        "svm_root_service_level": "performance",
        "intercluster_lifs": [
          {
            "name": "dev-ots-per01-01-icl01",
            "node": "dev-ots-per01-01",
            "port": "e0b",
            "address": "10.128.115.144",
            "netmask": "255.255.255.0"
          }
        ],
        "subscription_number": "A-S00003875",
        "created": "2019-02-22T03:38:38.867Z",
        "data_fcp_ports": [],
        "is_mcc": false,
        "created": "1995-09-07T10:40:52Z"
      }
    ]
  }
}
```


创建 **ONTAP** 集群

使用下表中的 API 创建 ONTAP 集群。

创建 ONTAP 集群时始终启用 iSCSI 服务。此外，如果基础架构支持 FCP 服务，也可以启用该服务。

HTTP 方法	路径	**Description**	**Parameters**
发布	` /v2.1/ontapclusters`	创建 ONTAP 集群。	无

所需的请求正文属性： `name` ， `uuid` ， `management_IP` ， `username` ， `password` ， `data_network_ip_cidr` ， `data_network_default_gateway` ， `intercluster_lifs` ， `zone`

如果启用了 FCP （使用 `services _available` FCP 属性），则需要 `data_fcp_ports `。

如果 `is_mcc` 为 true ，则需要 `mcc_partner_cluster` 。

- 请求正文示例： *

```
{
  "name": "clustername",
  "uuid": "49b6e08e-513a-11ea-b197-005056a4c0ef",
  "management_ip": "10.128.112.165",
  "username": "admin",
  "password": "ClusterPassword",
  "provisioning_state": "open",
  "data_network_ip_cidr": "10.96.112.0/24",
  "data_network_default_gateway": "10.96.112.1",
  "data_network_ports": [
    {
      "node_name": "clustername-01",
      "port_name": "e0c-112",
      "parent_port": "e0c"
    }
  ],
  "intercluster_lifs": [
    {
      "name": "clustername-01-icl01",
      "node": "clustername-01",
      "port": "e0b",
      "address": "10.128.112.222",
      "netmask": "255.255.255.0"
    }
  ],
  "svm_root_service_level": "extreme",
  "zone": "MyZone",
  "subscription_number": "",
  "services_available": {
    "fcp": false,
    "iscsi": true,
    "nas": true
  },
  "data_fcp_ports": [
  ],
  "is_mcc": false,
}
```


- 响应正文示例： *


```
{
    "status": {
        "user_message": "Okay. New resource created.",
        "verbose_message": "",
```

```
        "code": 201
    },
    "result": {
        "returned_records": 1,
        "records": [
            {
                "id": "5ef155b8f5591100010a75c5",
                "name": "clustername",
                "region": "MyRegion",
                "zone": "MyZone",
                "uuid": "49b6e08e-513a-11ea-b197-005056a4c0ef",
                "management_ip": "10.128.112.165",
                "username": "admin",
                "services_available": {
                    "fcp": false,
                    "iscsi": true,
                    "nas": true
                },
                "provisioning_state": "open",
                "data_network_ports": [
                    {
                        "node_name": "clustername-01",
                        "port_name": "e0c-112",
                        "parent_port": "e0c"
                    }
                ],
                "data_network_ip_cidr": "10.96.112.0/24",
                "data_network_default_gateway": "10.96.112.1",
                "svm_root_service_level": "extreme",
                "intercluster_lifs": [
                    {
                        "name": "clustername-01-icl01",
                        "node": "clustername-01",
                        "port": "e0b",
                        "address": "10.128.112.222",
                        "netmask": "255.255.255.0"
                    }
                ],
                "subscription_number": "",
                "created": "2020-06-23T01:07:04.563Z",
                "data_fcp_ports": [],
                "is_mcc": false,
                "mcc_partner_cluster": "5d2fb0fb4f47df00015274e3",
                "created": "1995-09-07T10:40:52Z"
            }
        ]
```

```
      }
}
```

修改 **ONTAP** 集群

使用以下列出的方法修改 ONTAP 集群。

HTTP 方法	路径	**Description**	**Parameters**
`PUT`	` /v2.1/ontapclusters/ { id } `	修改 ID 标识的 ONTAP 集群的详细信息。	`id` （ `string` ）：ONTAP 集群的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
{
  "name": "clustername",
  "uuid": "49b6e08e-513a-11ea-b197-005056a4c0ef",
  "management_ip": "10.128.112.165",
  "username": "admin",
  "password": "ClusterPassword",
  "provisioning_state": "open",
  "data_network_ip_cidr": "10.96.112.0/24",
  "data_network_default_gateway": "10.96.112.1",
  "data_network_ports": [
    {
      "node_name": "dev-ots-syd01-01",
      "port_name": "e0c-112",
      "parent_port": "e0c"
    }
  ],
  "intercluster_lifs": [
    {
      "name": "dev-ots-syd01-01-icl01",
      "node": "dev-ots-syd01-01",
      "port": "e0b",
      "address": "10.128.112.222",
      "netmask": "255.255.255.0"
    }
  ],
  "svm_root_service_level": "standard",
  "zone": "MyZone",
  "subscription_number": "",
  "services_available": {
    "fcp": false,
    "iscsi": true,
    "nas": false
  },
  "data_fcp_ports": [
  ]
}
```


- 响应正文示例：*


```
{
    "status": {
        "user_message": "Okay. Accepted for processing.",
        "verbose_message": "",
        "code": 202
```

```
    },
    "result": {
        "returned_records": 1,
        "records": [
            {
                "id": "5ef155b8f5591100010a75c5",
                "name": "clustername",
                "region": "MyRegion",
                "zone": "MyZone",
                "uuid": "49b6e08e-513a-11ea-b197-005056a4c0ef",
                "management_ip": "10.128.112.165",
                "username": "admin",
                "services_available": {
                    "fcp": false,
                    "iscsi": true,
                    "nas": true
                },
                "provisioning_state": "open",
                "data_network_ports": [
                    {
                        "node_name": "dev-ots-syd01-01",
                        "port_name": "e0c-112",
                        "parent_port": "e0c"
                    }
                ],
                "data_network_ip_cidr": "10.96.112.0/24",
                "data_network_default_gateway": "10.96.112.1",
                "svm_root_service_level": "standard",
                "intercluster_lifs": [
                    {
                        "name": "dev-ots-syd01-01-icl01",
                        "node": "dev-ots-syd01-01",
                        "port": "e0b",
                        "address": "10.128.112.222",
                        "netmask": "255.255.255.0"
                    }
                ],
                "subscription_number": "",
                "created": "2020-06-23T01:07:04.563Z",
                "data_fcp_ports": [],
                "is_mcc": false,
                "mcc_partner_cluster": "5d2fb0fb4f47df00015274e3",
                "created": "1995-09-07T10:40:52Z"
            }
        ]
    }
```

```
}
```

删除 **ONTAP** 集群

使用下表中列出的方法删除 ONTAP 集群。

HTTP 方法	路径	**Description**	**Parameters**
d删除	` /v2.1/ontapclusters/ ｛ id ｝ `	删除 ID 标识的 ONTAP 集群。	`id （ string ）：` ONTAP 集群的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
No content for succesful delete
```

StorageGRID 实例

使用下表中列出的方法设置和管理 StorageGRID 实例以支持对象存储。

HTTP 方法	路径	**Description**
获取	` /v2.1/storageggridinstances`	检索 StorageGRID 实例。
发布	` /v2.1/storageggridinstances`	创建新的 StorageGRID 实例以托管对象存储。
获取	` /v2.1/storageggridinstances/ ｛ id ｝ `	按 ID 检索 StorageGRID 实例。
`PUT`	` /v2.1/storageggridinstances/ ｛ id ｝ `	更新 StorageGRID 实例。

StorageGRID 实例属性

下表列出了 StorageGRID 实例属性。

属性	**Type**	**Description**
`id`	string	StorageGRID 实例的唯一标识符。
名称	string	StorageGRID 实例名称。
`admin_rest_uri`	字符串（ $uri ）	StorageGRID 管理节点端点。

属性	Type	Description
`s 3_Endpoints`	字符串（$uri）	StorageGRID 端点。例如：[`"<a href="https://s3.examplegrid.com"" class="bare">https://s3.examplegrid.com"</a>，]
`grid_username`	string	StorageGRID 用户名。
`grid_password`	string	StorageGRID 密码。
`tenant_username`	string	StorageGRID 租户用户名。
`tenant_password`	string	StorageGRID 租户密码。
`ssubscription_number`	string	Zuora 订阅编号。

检索 **StorageGRID** 实例

使用下表中列出的方法检索 StorageGRID 实例。

HTTP 方法	路径	**Description**	**Parameters**
获取	`/v2.1/storageggridinstances`	检索 StorageGRID 实例。	无

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5e3ba2840271823644cb8ab6",
        "name": "NSE StorageGRID Dev1",
        "user_id": "5bbee380a2df7a04d43acaee",
        "admin_rest_uri": "https://sggmi-dev.dev.ausngs.netapp.au",
        "s3_endpoints": [
          "https://sgs3.dev.ausngs.netapp.au"
        ],
        "subscription_number": "A-S00004566"
      }
    ]
  }
}
```


按 **ID** 检索 **StorageGRID** 实例

使用下表中列出的方法按 ID 检索 StorageGRID 实例。

HTTP 方法	路径	**Description**	**Parameters**
获取	`/v2.1/storageggridinstances/｛id｝`	按 ID 检索 StorageGRID 实例。	`id （ string ）：` StorageGRID 实例的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5e3ba2840271823644cb8ab6",
        "name": "NSE StorageGRID Dev1",
        "user_id": "5bbee380a2df7a04d43acaee",
        "admin_rest_uri": "https://sggmi-dev.dev.ausngs.netapp.au",
        "s3_endpoints": [
          "https://sgs3.dev.ausngs.netapp.au"
        ],
        "subscription_number": "A-S00004566"
      }
    ]
  }
}
```


按 **ID** 创建 **StorageGRID** 实例

使用下表中列出的方法按 ID 创建 StorageGRID 实例。

HTTP 方法	路径	**Description**	**Parameters**
发布`	` /v2.1/storageggridinstances/｛id｝`	按 ID 检索 StorageGRID 实例。	`id` （字符串）：StorageGRID 实例的唯一标识符。

所需的请求正文属性： `none`

- 请求正文示例： *

```
{
  "name": "Grid1",
  "admin_rest_uri": "https://examplegrid.com",
  "s3_endpoints": [
    "https://s3.examplegrid.com",
    "https://s3.location.company.com"
  ],
  "grid_username": "root",
  "grid_password": "string",
  "tenant_username": "root",
  "tenant_password": "string",
  "subscription_number": "A-S00003969"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "string",
    "verbose_message": "string",
    "code": "string"
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5d2fb0fb4f47df00015274e3",
        "name": "Grid1",
        "admin_rest_uri": "https://examplegrid.com",
        "user_id": "5d2fb0fb4f47df00015274e3",
        "s3_endpoints": [
          "https://s3.examplegrid.com",
          "https://s3.location.company.com"
        ],
        "subscription_number": "A-S00003969"
      }
    ]
  }
}
```

按 ID 修改 StorageGRID 实例

使用下表中列出的方法按 ID 修改 StorageGRID 实例。

HTTP 方法	路径	Description	Parameters
PUT	`/v2.1/storageggridinstances/｛id｝`	按 ID 修改 StorageGRID 实例。	id （ string ）：StorageGRID 实例的唯一标识符。

所需请求正文属性： none

- 请求正文示例： *

```
{
  "name": "Grid1",
  "admin_rest_uri": "https://examplegrid.com",
  "s3_endpoints": [
    "https://s3.examplegrid.com",
    "https://s3.location.company.com"
  ],
  "grid_username": "root",
  "grid_password": "string",
  "tenant_username": "root",
  "tenant_password": "string",
  "subscription_number": "A-S00003969"
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "string",
    "verbose_message": "string",
    "code": "string"
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "5d2fb0fb4f47df00015274e3",
        "name": "Grid1",
        "admin_rest_uri": "https://examplegrid.com",
        "user_id": "5d2fb0fb4f47df00015274e3",
        "s3_endpoints": [
          "https://s3.examplegrid.com",
          "https://s3.location.company.com"
        ],
        "subscription_number": "A-S00003969"
      }
    ]
  }
}
```


服务级别

使用下表中列出的方法检索，创建，修改和删除服务级别。

HTTP 方法	路径	**Description**
获取	` /v2.1/ServiceLevels`	检索所有服务级别。
获取	` /v2.1/ServiceLevers/ { id } `	按 ID 检索服务级别。
发布	` /v2.1/ServiceLevels`	创建新的服务级别。
PUT	` /v2.1/ServiceLevers/ { id } `	按 ID 修改服务级别详细信息。
d删除	` /v2.1/ServiceLevers/ { id } `	删除服务级别。

服务级别属性

下表列出了服务级别属性。

属性	**Type**	**Description**
id	string	服务级别的唯一标识符。
名称	string	服务级别名称。

属性	Type	Description
问题描述`	string	服务级别的问题描述。
policy_name	string	服务级别服务质量（QoS）名称。允许的值： NSE_value ，nse_standard ， nse_performance 和 nse_Extreme 。
可用	布尔值	指示服务级别是否可供使用。
祖父系	布尔值	指示服务级别是否已停用。
pease_IOPS_per_TB	整型	每 TiB 的最大可能 IOPS 。
expected_IOPS_per_TB	整型	每 TiB 的最小预期 IOPS 。
abory_min_IOPS	整型	当预期 IOPS 小于此值时用作覆盖的绝对最小 IOPS 。
pease_IOPS_allocation	string	峰值 IOPS 分配。允许值：allocated_space 和 used_space 。
io_block_size_KB	整型	I/O 块大小（KiB）。
min_size_GB	整型	最小大小（GiB）。
max_size_GB	整型	最大大小（GiB）。
max_pease_IOPS	整型	服务级别的最大峰值 IOPS 。
max_expected_IOPS	整型	最大预期 IOPS 。
autogrow_max_percent	整型	自动增长最大百分比。

属性	Type	Description
ontap_aggregates	–	ONTAP 聚合的列表。ONTAP 聚合包括： • cluster_UIID ： ONTAP 集群 UUID • aggr_name ： ONTAP 聚合名称 • aggr_uuid ： ONTAP 聚合 UUID • node_name ： ONTAP 集群节点名称 例如： "ontap_aggregates": [{ "cluster_uuid": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "aggr_name": "string", "aggr_uuid": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "node_name": "node01" }]

属性	Type	Description
primary_volume_defaults	–	• snapshot_auto_delete_target_free_space （整型）：此选项指定必须停止自动删除 Snapshot 副本的可用空间百分比。 • auto_size_mode （字符串）：卷的自动调整大小模式。允许的值： off ， grow ， grow_shrink ，例如： `"primary_volume_defaults": { "snapshot_auto_delete_target_free_space": 3, "auto_size_mode": "grow_shrink"`

检索服务级别

使用下表中列出的方法检索所有服务级别。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/ServiceLevels`	检索所有服务级别。	无

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 3 records.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "total_records": 3,
    "records": [
```

```
      {
        "name": "standard",
        "description": "Best suited for general purpose workloads",
        "slo": "1000IOPS/TB",
        "min_size": 137438953472,
        "io_block_size_kb": 32,
        "min_size_gb": 10,
        "max_size_gb": 40960,
        "min_iops": 100,
        "peak_iops_per_tb": 1000,
        "expected_iops_per_tb": 700,
        "max_peak_iops": 40000,
        "max_expected_iops": 28000,
        "max_peak_throughput": 1250,
        "max_expected_throughput": 875
      },
      {
        "name": "extreme",
        "description": "Best suited for performance-critical workloads",
        "slo": "12000IOPS/TB",
        "min_size": 91625968981,
        "io_block_size_kb": 32,
        "min_size_gb": 10,
        "max_size_gb": 10240,
        "min_iops": 500,
        "peak_iops_per_tb": 12000,
        "expected_iops_per_tb": 8000,
        "max_peak_iops": 120000,
        "max_expected_iops": 60000,
        "max_peak_throughput": 3750,
        "max_expected_throughput": 1875
      },
      {
        "name": "premium",
        "description": "Best suited for databases and high performance
workloads",
        "slo": "4000IOPS/TB",
        "min_size": 137438953472,
        "io_block_size_kb": 32,
        "min_size_gb": 10,
        "max_size_gb": 10240,
        "min_iops": 300,
        "peak_iops_per_tb": 4000,
        "expected_iops_per_tb": 3000,
        "max_peak_iops": 40000,
        "max_expected_iops": 30000,
```

```
        "max_peak_throughput": 1250,
        "max_expected_throughput": 937
      }
    ]
  }
}
```

按名称检索服务级别

使用下表中列出的方法按名称检索服务级别。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/ServiceLevers/ ｛ name ｝ `	按名称检索服务级别。	`name` （ `string` ）：服务级别的名称。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "name": "premium",
        "description": "Best suited for databases and high performance
workloads",
        "slo": "4096IOPS/TB",
        "min_size": 137438953472,
        "io_block_size_kb": 32,
        "min_size_gb": 10,
        "max_size_gb": 10240,
        "min_iops": 300,
        "peak_iops_per_tb": 4096,
        "expected_iops_per_tb": 3000,
        "max_peak_iops": 40000,
        "max_expected_iops": 30000,
        "max_peak_throughput": 1250,
        "max_expected_throughput": 937
      }
    ]
  }
}
```


创建服务级别

使用下表中列出的方法创建服务级别。

HTTP 方法	路径	Description	Parameters
发布	` /v2.1/ServiceLevels`	创建服务级别。	无

所需的请求正文属性： `name` ， `policy_name`

- 请求正文示例： *

```
{
  "name": "MyServiceLevelName",
  "description": "My new service level description",
  "policy_name": "nse_value",
  "available": true,
  "grandfathered": false,
  "peak_iops_per_tb": 1000,
  "expected_iops_per_tb": 700,
  "absolute_min_iops": 100,
  "peak_iops_allocation": "allocated_space",
  "io_block_size_kb": 32,
  "min_size_gb": 10,
  "max_size_gb": 40960,
  "max_peak_iops": 20000,
  "max_expected_iops": 5000,
  "autogrow_max_percent": 3,
  "ontap_aggregates": [
    {
      "cluster_uuid": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "aggr_name": "string",
      "aggr_uuid": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "node_name": "node01"
    }
  ],
  "primary_volume_defaults": {
    "snapshot_auto_delete_target_free_space": 3,
    "auto_size_mode": "grow_shrink"
  }
}
```


- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. New resource created.",
    "verbose_message": "",
    "code": 201
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "name": "MyServiceLevelName",
        "description": "My new service level description",
        "slo": "1000IOPS/TB",
        "min_size": 0,
        "io_block_size_kb": 32,
        "min_size_gb": 10,
        "max_size_gb": 40960,
        "min_iops": 100,
        "peak_iops_per_tb": 1000,
        "expected_iops_per_tb": 700,
        "max_peak_iops": 20000,
        "max_expected_iops": 5000,
        "max_peak_throughput": 625,
        "max_expected_throughput": 156
      }
    ]
  }
}
```


修改服务级别

使用下表中列出的方法修改服务级别。

HTTP 方法	路径	Description	Parameters
`PUT`	`/v2.1/ServiceLevers/{name}`	修改服务级别的详细信息。	`name` （ `string` ）：服务级别的名称。

所需的请求正文属性： `none`

- 请求正文示例： *

```
{
  "name": "MyNewServiceLevelName",
  "description": "Service level description",
  "policy_name": "nse_value",
  "available": false,
  "grandfathered": false,
  "peak_iops_per_tb": 1000,
  "expected_iops_per_tb": 700,
  "absolute_min_iops": 100,
  "peak_iops_allocation": "allocated_space",
  "io_block_size_kb": 32,
  "min_size_gb": 10,
  "max_size_gb": 40960,
  "max_peak_iops": 20000,
  "max_expected_iops": 5000,
  "autogrow_max_percent": 3,
  "ontap_aggregates": [
    {
      "cluster_uuid": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "aggr_name": "string",
      "aggr_uuid": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "node_name": "node01"
    }
  ],
  "primary_volume_defaults": {
    "snapshot_auto_delete_target_free_space": 3,
    "auto_size_mode": "grow_shrink"
  }
}
```

- 响应正文示例： *

```
TBA
```

按 **ID** 删除服务级别

使用下表中列出的方法按 ID 删除服务级别。

HTTP 方法	路径	**Description**	**Parameters**
d删除	` /v2.1/ServiceLevers/ ｛ name ｝ `	删除 ID 标识的服务级别。	name （ string ）：服务级别的名称。

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
No content for succesful delete
```

服务请求

使用下表中列出的方法创建和检索服务请求。

HTTP 方法	路径	Description
获取	` /v2.1/tenant/ ｛ tenant_id ｝ /servicerequests`	检索服务请求。
获取	` /v2.1/tenant/ ｛ tenant_id ｝ / 服务要求 / ｛ id ｝ `	按 ID 检索服务请求。
发布	` /v2.1/tenant/ ｛ tenant_id ｝ /servicereques/`	创建服务请求。
获取	` /v2.1/tenant/ ｛ tenant_id ｝ /servicerequies/categories`	检索服务请求类别。

服务请求属性

下表列出了服务请求属性。

属性	Type	Description
`ID`	string	服务请求的标识符。例如： `sQ0035952014` 。
`sSubject`	string	服务请求的主题。
注释	string	对服务请求的注释
类别	string	请求的类别：备份，灾难恢复，技术，其他或订阅。
优先级	string	服务请求的优先级：极低，低，正常，高或紧急。
`ssubscription`	string	Zuora 订阅编号。

属性	Type	Description
承诺	–	订阅承诺详细信息，包括： • 服务级别（字符串）：服务级别：标准，高级或至尊。 • new_commitment （整型）：需要新的总容量。
分区	string	分区名称。
`ssubtenant_id`	string	子租户标识符。
`storage_object_type`	string	存储对象类型：文件服务器或块存储。
`storage_object_id`	string	存储对象的标识符。例如：`5d2fb0fb4f47df00015274e3`
`sservice_type`	string	服务类型：文件服务，块存储或对象存储。

检索服务请求

使用下表中列出的方法检索指定租户的服务请求类别。

HTTP 方法	路径	Description	Parameters
获取`	` /v2.1/tenant/ { tenant_id } /servicerequests`	检索服务请求。	`tenant_id`：（可选）返回指定租户的服务请求。`offset` 和 `limit` — 请参见 "通用分页"

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "total_records": 34,
    "sort_by": "created",
    "order_by": "desc",
    "offset": 6,
    "limit": 1,
    "records": [
      {
        "id": "SRQ0035952014",
        "subject": "DR Failover - fileserver",
        "description": "catgory:Disaster Recovery Failover \n   subtenant:
DefaultSubtenants2 \n  region: au-east2 \n zone: au-east2-a \n
fileserver: Demotsysserv1 \n tenant:MyOrg \n comments:comments",
        "priority": "Urgent",
        "status": "New",
        "createdDate": "2020-05-22T04:23:12+0000",
        "updatedDate": "2020-05-22T04:23:12+0000"
      }
    ]
  }
}
```


按 **ID** 检索服务请求

使用下表中列出的方法按服务请求 ID 检索服务请求。

HTTP 方法	路径	**Description**	**Parameters**
获取	` /v2.1/tenant/ ｛tenant_id｝ / 服务要求 / ｛id｝ `	按 ID 检索服务请求。	• `tenant_id`： 租户 ID • `id`： 服务请求 ID 例如：SRQ0035952014

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *


```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "SRQ0035952014",
        "subject": "DR Failover - fileserver",
        "description": "catgory:Disaster Recovery Failover \n   subtenant:
DefaultSubtenants2 \n  region: au-east2 \n zone: au-east2-a \n
fileserver: Demotsysserv1 \n tenant:MyOrg \n comments:comments",
        "priority": "Urgent",
        "status": "New",
        "createdDate": "2020-05-22T04:23:12+0000",
        "updatedDate": "2020-05-22T04:23:12+0000"
      }
    ]
  }
}
```


创建服务请求

使用下表中列出的方法创建服务请求。

HTTP 方法	路径	Description	Parameters
发布	` /v2.1/tenant/ ｛ tenant_id ｝ /servicerequies/categories `	创建服务请求。	`tenant_id`： 租户标识符。

所需请求正文属性：所需属性取决于服务请求的类别。下表列出了请求正文属性。

类别	Required
订阅。	`ssubscription` and `commitment`

类别	Required
灾难恢复	storage_object_type ， ssubtenant_id 和 storage_object_id
技术	ssubtenant_id 和 sservice_type 如果 sservice_type 是文件服务或块存储，则需要区域。
其他	分区

- 请求正文示例： *

```
{
  "subject": "string",
  "comment": "string",
  "category": "subscription",
  "priority": "Normal",
  "subscription": "A-S00003969",
  "commitment": {
    "service_level": "standard",
    "new_commitment": 10
  },
  "zone": "au-east1-a",
  "subtenant_id": "5d2fb0fb4f47df00015274e3",
  "storage_object_type": "fileserver",
  "storage_object_id": "5d2fb0fb4f47df00015274e3",
  "service_type": "File Services"
}
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "string",
    "verbose_message": "string",
    "code": "string"
  },
  "result": {
    "returned_records": 1,
    "records": [
      {
        "id": "string",
        "subject": "string",
        "description": "string",
        "status": "New",
        "priority": "Normal",
        "createdDate": "2020-05-12T03:18:25+0000",
        "UpdatedDate": "2020-05-12T03:18:25+0000"
      }
    ]
  }
```

检索服务请求类别

下表列出了指定租户的检索服务请求类别。

HTTP 方法	路径	Description	Parameters
获取	` /v2.1/tenant/ ｛ tenant_id } /servicerequies/categories `	检索服务请求类别。	`tenant_id` ： （可选）返回指定租户的服务请求。

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 5 records.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 5,
    "records": [
      {
        "key": "dr",
        "value": "Disaster Recovery Failover"
      },
      {
        "key": "technical",
        "value": "Technical Issue"
      },
      {
        "key": "other",
        "value": "Other"
      },
      {
        "key": "subscription",
        "value": "Subscription Management"
      },
      {
        "key": "backup",
        "value": "Backup Restore"
      }
    ]
  }
}
```


作业

使用作业 API 查看作业详细信息。有关使用作业的详细信息，请参见作业对象 API 和异步进程。

检索作业

使用下表中列出的方法检索所有作业或部分作业。指定 `tenant_id` 将仅返回属于该租户的作业。

HTTP 方法	路径	Description	Parameters
获取	` v2.1/jobs`	检索作业。	`tenant_id`：（可选）返回属于指定租户的块存储。`offset` 和 `limit` — 请参见 "通用分页"

所需的请求正文属性： `none`

- 请求正文示例： *

```
none
```

- 响应正文示例： *

```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
    "code": 200
  },
  "result": {
    "returned_records": 1,
    "total_records": 2625,
    "sort_by": "created",
    "order_by": "desc",
    "offset": 0,
    "limit": 1,
    "records": [
      {
        "id": "5ed72c8c6342e90001439d54",
        "action": "create",
        "job_summary": "Create request is successfully submitted",
        "created": "2020-06-03T04:52:28.478Z",
        "updated": "2020-06-03T04:52:32.636Z",
        "object_id": "5ed72c8c6342e90001439d55",
        "object_type": "sg_buckets",
        "object_name": "test1234",
        "status": "successful",
        "status_detail": "Creation of s3 bucket 'test1234' completed
successfully.",
        "last_error": "",
        "user_id": "5e85225af038943eb4b74684",
        "job_tasks": [
          {
            "id": "5ed72c8c6342e90001439d57",
```

```
            "job_id": "5ed72c8c6342e90001439d54",
            "action": "create",
            "driver": "storagegrid_ansible",
            "object_id": "5ed72c8c6342e90001439d55",
            "object_type": "sg_buckets",
            "resource_type": "sg_bucket",
            "status": "successful",
            "status_detail": "Worker completed task successfully.",
            "last_error": "",
            "user_id": "5e85225af038943eb4b74684",
            "request_payload": {
              "grid_account_id": "05336917559886003354",
              "grid_admin_base_url": "https://sggmi-
dev.dev.ausngs.netapp.au/api/v3",
              "org_password": "netapp01",
              "org_username": "root",
              "s3_bucket_name": "test1234"
            }
          }
        ]
      }
    ]
  }
}
```


按作业 **ID** 检索作业

使用下表中列出的方法按 ID 检索作业。

HTTP 方法	路径	**Description**	**Parameters**
获取	`/v2.1/jobs/｛id｝`	按 ID 检索作业。	`id` （ `string` ）：作业的唯一标识符。

所需的请求正文属性： 作业标识符

- 请求正文示例： *

```
none
```

- 响应正文示例： *


```
{
  "status": {
    "user_message": "Okay. Returned 1 record.",
    "verbose_message": "",
```

```
    "code": 200
  },
  "result": {
    "total_records": 1,
    "records": [
      {
        "id": "5e66f18e09a74c0001b89640",
        "action": "create",
        "job_summary": "Create S3 bucket for Sub Tenant",
        "created": "2020-03-10T01:46:54.097Z",
        "updated": "2020-03-10T01:46:57.664Z",
        "object_id": "5e66f18e09a74c0001b89641",
        "object_type": "sg_buckets",
        "status": "successful",
        "status_detail": "Creation of s3 bucket 'mys3bucket' completed
successfully.",
        "last_error": "",
        "user_id": "5bbee380a2df7a04d43acaee",
        "job_tasks": [
          {
            "id": "5e66f18e09a74c0001b89642",
            "job_id": "5e66f18e09a74c0001b89640",
            "action": "create",
            "driver": "storagegrid_ansible",
            "object_id": "5e66f18e09a74c0001b89641",
            "object_type": "sg_buckets",
            "resource_type": "sg_bucket",
            "status": "successful",
            "status_detail": "Worker completed task successfully.",
            "last_error": "",
            "user_id": "5bbee380a2df7a04d43acaee",
            "request_payload": {
              "grid_account_id": "47490102387197219062",
              "grid_admin_base_url": "https://sggmi-
dev.dev.ausngs.netapp.au/api/v3",
              "org_password": "netapp01",
              "org_username": "root",
              "s3_bucket_name": "mys3bucket"
            }
          }
        ]
      }
    ]
  }
}
```

作业对象 API 和异步进程

某些 API 调用（尤其是用于添加或修改资源的 API 调用）的完成时间可能比其他调用更长。NetApp 服务引擎会异步处理这些长时间运行的请求。

发出异步运行的 API 调用后， HTTP 响应代码 202 表示此请求已成功验证并被接受，但尚未完成。此请求将作为后台任务进行处理，在对客户端进行初始 HTTP 响应后，此任务将继续运行。响应包括作业对象锁定请求，包括其唯一标识符。

正在查询与 API 请求关联的作业对象

HTTP 响应中返回的作业对象包含多个属性。您可以查询 state 属性以确定请求是否成功完成。作业对象可以处于以下状态之一：

- 正常
- 警告
- Partial_failures
- error

在轮询作业对象以检测任务的终端状态时，可以使用两种方法：成功或失败：

- 标准轮询请求：立即返回当前作业状态。
- 长轮询请求：当作业状态变为 normal ， error 或 partial_failures 时。

异步请求的步骤

您可以使用以下高级操作步骤完成异步 API 调用。

1. 问题描述异步 API 调用。
2. 接收表示已成功接受请求的 HTTP 响应 202 。
3. 从响应正文中提取作业对象的标识符。
4. 在环路中，等待作业对象达到终端状态 normal ， error 或 partial_failures 。
5. 验证作业的终端状态并检索作业结果。

Copyright Information

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

Trademark Information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.